

Introduction to programming

Labor08

Revision

- ▶ Recursive functions

Homework 1

<https://viskillz.inf.unideb.hu/prog/#/2022?week=P1041>

- ▶ Write a function named *count_of_squares()* that returns how many elements in a list are **perfect squares**.
- ▶ In the *main()* function:
 - ▶ Read list elements until the given condition (EOF, n test cases, or empty line)
 - ▶ Call *count_of_squares()* and print the result for each list.

Homework1

Sample Input

- ▶ 63 64 14 6 10 6 39 37 71
- ▶ 71 9 81
- ▶ 82 65 71 79 46 88

Output for Sample Input

- ▶ 1
- ▶ 2
- ▶ 0

Count of squares

```
def count_of_squares(numbers: list[int]) -> int:  
    count = 0  
    for number in numbers:  
        if number == int(math.sqrt(number)) ** 2:  
            count += 1  
  
    return count
```

Count of squares – Until EOF

```
import sys
import math

def count_of_squares(numbers: list[int]) -> int:
    count = 0
    for number in numbers:
        if number == int(math.sqrt(number) ** 2):
            count += 1
    return count

def main():
    for line in sys.stdin:
        numbers = line.split(" ")
        lists = []
        for number in numbers:
            lists.append(int(number))
        print(count_of_squares(lists))

if __name__ == '__main__':
    main()
```

Count of squares – N test cases

```
import math

def count_of_squares(numbers: list[int]) -> int:
    count = 0
    for number in numbers:
        if number == int(math.sqrt(number) ** 2):
            count += 1
    return count

def main():
    n = int(input())
    for i in range(n):
        numbers = input().split(" ")
        lists = []
        for number in numbers:
            lists.append(int(number))
        print(count_of_squares(lists))

if __name__ == '__main__':
    main()
```

Count of squares – Until an empty line

```
import sys
import math

def count_of_squares(numbers: list[int]) -> int:
    count = 0
    for number in numbers:
        if number == int(math.sqrt(number) ** 2):
            count += 1
    return count

def main():
    for line in sys.stdin:
        if line.strip() == "":
            break
        numbers = line.split(" ")
        lists = []
        for number in numbers:
            lists.append(int(number))
        print(count_of_squares(lists))

if __name__ == '__main__':
    main()
```


Homework 2

<https://viskillz.inf.unideb.hu/prog/#/2022?week=P1051>

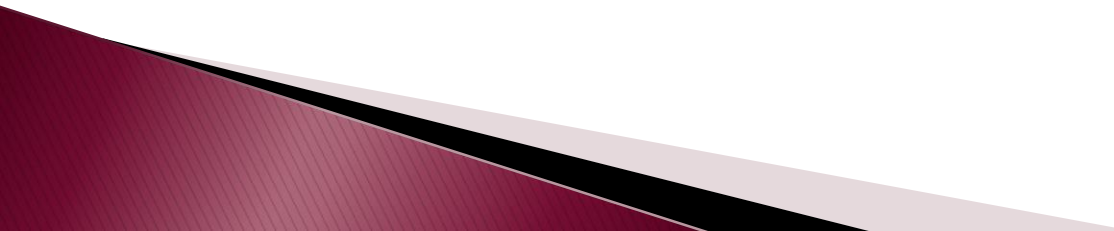
- ▶ Write a function named *delete_even_digits()* that removes all **even digit characters** from a given string and returns the result.
- ▶ In the *main()* function:
- ▶ Read strings until EOF / n test cases / empty line
- ▶ Call *delete_even_digits()* and print the modified string.

Homework2

Sample Input

- ▶ 0123456789
- ▶ a2bc4d1ef3gh
- ▶ aB1cD2eF3gH4

Output for Sample Input

- ▶ 13579
 - ▶ abcd1ef3gh
 - ▶ aB1cDeF3gH
- 
- A decorative gradient bar at the bottom left of the slide, transitioning from dark red to light grey.

Delete even digits

```
def delete_even_digits(original: str) -> str:  
    r = ''  
    for c in original:  
        if c not in '02468':  
            r = r + c  
    return r
```

Delete even digits

```
def delete_even_digits(original: str) -> str:  
    return "".join([c for c in original  
                      if c not in "02468"])
```

Delete even digits – Until EOF

```
import sys
```

```
def delete_even_digits(original: str) -> str:  
    return "".join([c for c in original  
                      if c not in "02468"])
```

```
def main():  
    for line in sys.stdin:  
        print(delete_even_digits(line.strip()))
```

```
if __name__ == '__main__':  
    main()
```

Delete even digits – N test cases

```
def delete_even_digits(original: str) -> str:
    return "".join([c for c in original
                     if c not in "02468"])

def main():

    n = int(input())
    for i in range(n):
        line = input()
        print(delete_even_digits(line))

if __name__ == '__main__':
    main()
```

Delete even digits – Until an empty line

```
import sys

def delete_even_digits(original: str) -> str:
    return "".join([c for c in original
                     if c not in "02468"])

def main():
    for line in sys.stdin:
        if line.strip() == "":
            break
        print(delete_even_digits(line.strip()))
```

Homework3

<https://viskillz.inf.unideb.hu/prog/#/?week=P1032>

- ▶ Write a function called *Pythagorean* that calculates the **hypotenuse** of a right triangle from its two sides (*a* and *b*).
- ▶ In the *main()* function:
- ▶ Read pairs of numbers until EOF / n test cases / empty line. (a b → in one line, separated by the space character)
- ▶ Call *Pythagorean()* and print the hypotenuse rounded to two decimal places.

Homework3

Sample Input

- ▶ 86.3 50.47
- ▶ 59.73 39.11
- ▶ 57.4 55.74
- ▶ 61.91 52.71
- ▶ 11.15 87.08

Output for Sample Input

- ▶ 99.97
- ▶ 71.40
- ▶ 80.01
- ▶ 81.31
- ▶ 87.79

Pythagorean() function

```
import math
```

```
def Pythagorean(a: float, b: float) -> float:  
    return math.sqrt(a**2 + b**2)
```

Pythagorean() function – Until EOF

```
import math
import sys

def Pythagorean(a: float, b: float) -> float:
    return math.sqrt(a**2 + b**2)

def main():
    for line in sys.stdin:
        numbers=line.split()
        a=float(numbers[0])
        b=float(numbers[1])
        print('{0:.2f}'.format(Pythagorean(a,b)))

if __name__ == '__main__':
    main()
```

Pythagorean() function – N test cases

```
import math
import sys

def Pythagorean(a: float, b: float) -> float:
    return math.sqrt(a**2 + b**2)

def main():
    n = int(input())
    for i in range(n):
        numbers = input().split()
        a=float(numbers[0])
        b=float(numbers[1])
        print('{0:.2f}'.format(Pythagorean(a,b)))

if __name__ == '__main__':
    main()
```

Pythagorean() function – Until an empty line

```
import math
import sys

def Pythagorean(a: float, b: float) -> float:
    return math.sqrt(a**2 + b**2)

def main():
    for line in sys.stdin:
        if line.strip() == "":
            break
        numbers=line.split()
        a=float(numbers[0])
        b=float(numbers[1])
        print('{0:.2f}'.format(Pythagorean(a,b)))

if __name__ == '__main__':
    main()
```

SET

- ▶ Sets are used to store multiple items in a single variable.
- ▶ The set type: **class set**
- ▶ In Python, the built-in **set** type represents a collection of unique, unordered elements.
- ▶ It supports the usual set operations:
 - insert an element,
 - remove an element,
 - intersection,
 - union,
 - difference
 - symmetric difference of sets.

SET

- ▶ Sets are written using **curly braces**/brackets.

Example

Create a Set:

- ▶ `s = {"apple", "banana", "cherry"}`
`print(s)`
- ▶ Properties of set elements:
 - **Unordered**: the items have no defined order and may appear in a different order each time.
 - **Unindexed**: elements cannot be accessed by index.
 - **Immutable as values**: you cannot change an existing element, but you can add or remove elements.
 - **No duplicates**: duplicate values are automatically removed.

SET

```
s = set(["hungarian", "english", "german"]) # 3 element
set
s = {"hungarian", "english", "german"}
s = set() # empty set, attention: {} means -> dictionary!

# queries: in, not in
print("english" in s)
print("japanese" not in s)

# modifiers: add, remove
s.add("italian")
s.remove("german")
```


SET operations

- ▶ $s1 \ \& \ s2$ intersection
- ▶ $s1 \ | \ s2$ union
- ▶ $s1 - s2$ difference
- ▶ $s1 \ \wedge \ s2$ delta/symmetric difference
- ▶ The $\&$ AND operator is the intersection: it looks for elements that are included in the first AND the second set.
- ▶ The $|$ OR operator is the union, looking for elements that are in the first OR the second set.
- ▶ The $-$ difference, takes the elements found in the second set from the first set.
- ▶ The delta operation is denoted by the $\wedge$ (hat, caret) operator. This returns elements that are in only in the first set or only in the second set (like the XOR operation).

Example

```
s1 = {"hungarian", "english", "german"}
s2 = set()
s2.add("italian")
s2.add("german")
print(s1)
print(s2)
print("-----")
# Set operations: intersection(&), union(|),
# difference(-), symmetric difference (delta, xor)
print(s1 & s2)
print(s1 | s2)
print(s1 - s2)
print(s1 ^ s2)
```

SET operations

SET operations	SET operations names
$ $	Union
$\&$	Intersection
$-$	Difference
$\wedge$	Symmetric Difference

```
s1 = {"hungarian", "english", "german"}
s2 = set()
s2.add("italian")
s2.add("german")
print(s1)
print(s2)
print("-----")
# Set operations: intersection(&), union(|),
difference(-), symmetric difference (^ delta, xor)
print(s1.intersection(s2))
print(s1 & s2)
print(s1.union(s2))
print(s1 | s2)
print(s1.difference(s2))
print(s1 - s2)
print(s1.symmetric_difference(s2))
print(s1 ^ s2)
```

SET operations

- ▶ **len(s)**: returns the length of the set s
- ▶ **x in s**: returns true if x is a member of set s
- ▶ **x not in s**: returns true if x is not an element of set s
- ▶ **s1.isdisjoint(s2)**: returns true if the intersection of s1 and s2 is an empty set (disjoint sets)
- ▶ **s1.issubset(s2)**:
 - **s1 <= s2**: returns true if s1 is a subset of s2
 - **s1 < s2**: returns true if s1 is a real subset of s2

SET operations

- ▶ **s.add(element)**: adds element to the set s.
- ▶ **s.remove(element)**: removes the element from set s. `KeyError` exception thrown if element not found in s.
- ▶ **s.discard(element)**: removes the element from s if it is in the set s.
- ▶ **s.pop()**: remove and return an arbitrary element from the set.
- ▶ **s.clear()**: clear all elements from the set.
- ▶ **s.copy()**: creates a copy of the set s

Exercise

- ▶ Read two integers from standard input that specify the sizes of two sets.
Fill both sets with random integers between 1 and 20.
- ▶ Then print:
 - the **union** of the two sets,
 - the **intersection** of the two sets,
 - the **difference** of the two sets,
 - the **symmetric difference** of the two sets.

Solution

```
import random
s1 = set()
s2 = set()
n = int(input("n = "))
m = int(input("m = "))
while len(s1) < n:
    s1.add(random.randint(1,20))
while len(s2) < m:
    s2.add(random.randint(1,20))
print(s1)
print(s2)
print("Union =", s1.union(s2))
print("Intersection =", s1.intersection(s2))
print("Difference =", s1.difference(s2))
print("Symmetric difference=", s1.symmetric_difference(s2))
```


Exercise

- ▶ Read the sizes of two sets from standard input.
- ▶ Fill one set with randomly generated **odd numbers** between 1 and 20, and the other set with randomly generated **even numbers** between 1 and 20.
- ▶ Then print the **union** of the two sets.

Solution

```
import random
even = set()
odd = set()
n = int(input("n = "))
m = int(input("m = "))
while len(even) < n:
    num = random.randint(1, 20)
    if num % 2 == 0:
        even.add(num)
while len(odd) < m:
    num = random.randint(1, 20)
    if num % 2 == 1:
        odd.add(num)
print(even)
print(odd)
print("Union =", even.union(odd))
```

Dictionary

- ▶ Dictionaries store data values in **key:value** pairs.
- ▶ In Python, the easiest way to create a dictionary, or **dict**-type container, is to list the **key:value** pairs between curly brackets, with a colon between them.
This is the most common form.
- ▶ The curly brackets may also be empty, which creates an empty dictionary that can be modified later.
- ▶ **Note:** You cannot create an empty set using curly brackets. Since there would be no difference between the empty set and the empty dictionary.
- ▶ If there is at least one element, it is clear which one is created: if there is a colon, then it is a dictionary, if there are no colons, then it is a set.

Dictionary

Create and print a dictionary:

```
shoppinglist = {  
    "apple": 2,  
    "pear": 1,  
    "peach": 5,  
}  
print(shoppinglist)
```

Result:

► {'apple': 2, 'pear': 1, 'peach': 5}

Dictionary

- ▶ You can create an empty dictionary either by calling the `dict()` constructor or by using an empty pair of curly braces: `{}`.
- ▶ Dictionary elements can be added or accessed using the **indexing operator** (`[]`).

```
shoppinglist = dict()           # or {}  
shoppinglist["apple"] = 2  
shoppinglist["pear"] = 1  
shoppinglist["peach"] = 5  
print(shoppinglist)
```

Dictionary

- ▶ A third option is to create a dictionary from a **list of items**.
- ▶ In this case, each item must be a **tuple containing a key-value pair**.

```
shoppinglist = dict([  
    ("apple", 2),  
    ("pear", 1),  
    ("peach", 5),  
)  
print(shoppinglist)
```

Access dictionary items

```
print(shoppinglist)
    #{'apple': 2, 'pear': 1, 'peach': 5}
print(shoppinglist["apple"])    # 2
print("apple" in shoppinglist)  # True
del shoppinglist["apple"]
shoppinglist["plum"] = 7
print(shoppinglist)
    #{'pear': 1, 'peach': 5, 'plum': 7}
try:
    print(shoppinglist["papaja"])
except KeyError as e:
    print("Not in it!")    # Not include
```

Access dictionary items

```
for key in shoppinglist: #keys  
    print(key)
```

```
for value in shoppinglist.values(): #values  
    print(value)
```

```
print("Sum:", sum(shoppinglist.values()))
```

```
for key, value in shoppinglist.items(): #key-values  
    print(f"{key}: {value} kg")
```


Access dictionary items

- ▶ If you only need the **values**, and the keys are not important, you can use the `dict.values()` method. This returns a **list** containing all values of the dictionary.
- ▶ The `sum()` function returns the sum of the values.
- ▶ To access **both keys and values** at the same time, the most convenient tool is the `dict.items()` method.
- ▶ It returns **tuple pairs** in the form (key, value) during iteration.
- ▶ Then the iteration will return tuple items, which are the items of the dictionary (key, value).

Exercise

- ▶ Write a program that reads words until an **empty string** is entered.
- ▶ Create a **frequency dictionary** that stores how many times each word was entered.

Solution

```
occurrence = {}
```

```
word = input()
```

```
while word != "":
```

```
    if word in occurrence:
```

```
        occurrence[word] += 1
```

```
    else:
```

```
        occurrence[word] = 1
```

```
    word = input()
```

```
for key, value in occurrence.items():
```

```
    print(f"{key}: {value}")
```

Monkey's habit

- ▶ <https://progcont.hu/progcont/100356/?pid=201536>

Monkey's habit

- ▶ Everyone knows that a monkey's favorite food is the delicious, sweet banana. It's no surprise that each monkey wants to collect as many of these tasty treats as possible.
- ▶ Write a program that reads **one monkey per line** until **end-of-file (EOF)**.
Each line contains the monkey's name and the number of bananas it has collected, in the following format:
 - ▶ `monkey_name;bananas_number`
- ▶ Your program should print all monkeys and the number of bananas they collected, sorted lexicographically in ascending order by the monkey's name, using the exact output format shown in the example!

Monkey's habit

► Sample Input

- Jambo;10
- Kongo;5
- Charlie;12
- Jambo;10
- Koko;14
- Kongo;10
- Jambo;5
- Charlie;8

► Output for Sample Input

- Charlie: 20
- Jambo: 25
- Koko: 14
- Kongo: 15

Solution

```
import sys

monkey = {}
for line in sys.stdin:
    data = line.split(';')
    if data[0] in monkey:
        monkey[data[0]] += int(data[1])
    else:
        monkey[data[0]] = int(data[1])

for key, value in sorted(monkey.items()):
    print(f'{key}: {value}')
```

Festival season

- ▶ <https://progcont.hu/progcont/100357/?pid=201540>

Festival season

- ▶ Concerts are the main source of income for popular music bands, who travel across the country to entertain their fans at various festivals.
- ▶ Write a program that reads data from standard input until **end-of-file (EOF)**.
- ▶ Each line contains the **name of a festival** followed by the **names of the bands performing there**, in the following format:
 - ▶ `festival_name:band_name[,band_name]...`
- ▶ Both `festival_name` and each `band_name` consist of a single string that contains **neither a colon nor a comma**.
- ▶ Your program should print each band's name in lexicographical ascending order, along with the number of festivals they have performed at, using the exact format shown in the example output!

Festival season

► Sample Input

- Sziget fesztival:Punnany Massif,Tankcsapda,Kowalsky meg a vega
- Campus fesztival:Tankcsapda,Fish!,Anna and the Barbies
- SZIN:Punnany Massif,Kowalsky meg a vega,Tankcsapda
- Volt:Anna and the Barbies,Tankcsapda,Kowalsky meg a vega

► Output for Sample Input

- Anna and the Barbies: 2
- Fish!: 1
- Kowalsky meg a vega: 3
- Punnany Massif: 2
- Tankcsapda: 4

Solution

```
import sys

bands = {}
for line in sys.stdin:
    data = line.split(':')
    names = data[1].strip().split(',')
    for i in range(len(names)):
        if names[i] in bands:
            bands[names[i]] += 1
        else:
            bands[names[i]] = 1

for key, value in sorted(bands.items()):
    print(f'{key}: {value}')
```

Lambda functions

- ▶ A lambda function is a small, anonymous function.
- ▶ A lambda function can take any number of arguments, but it can only have one expression.

Syntax:

- ▶ `lambda arguments : expression`
- ▶ The expression is evaluated and it's result is returned.

```
print((lambda x, y, z: (x + y + z) * 10)(1, 2, 3))  
60
```

```
lista = ["Jabba", "Han", "Luke"]  
print(sorted(lista, key = lambda item: -len(item)))  
['Jabba', 'Luke', 'Han']
```

Homework 1

Every day an apple keeps the doctor away

- ▶ <https://progcont.hu/progcont/100358/?pid=201544>

Homework 1

Every day an apple keeps the doctor away

- ▶ We have all heard the saying in the title many times **fruit is an essential part of a healthy diet.**
- ▶ Write a program that reads, line by line from standard input, the **names of fruits** and the **names of children who like that fruit.** Each line has the following format:
 - ▶ `fruit_name:child_name[,child_name]...`
- ▶ Both fruit names and children's names are **unique strings.** There are no duplicate entries such that the same child likes the same fruit more than once.
- ▶ Your program should print to standard output the names of the children in **lexicographically increasing order**, along with the **number of different fruits** each child likes, using the **exact format shown in the sample output.**

Homework 1

Every day an apple keeps the doctor away

Sample Input

```
apple:Pete,Kate  
plum:John  
banana:Kate,John,Charles  
pineapple:Hannah,Anna,Penny  
apricot:Fanny,Pete
```

Output for Sample Input

```
Anna: 1  
Charles: 1  
Fanny: 1  
Hannah: 1  
John: 2  
Kate: 2  
Penny: 1  
Pete: 2
```

Homework 2

New curriculum model

- ▶ <https://progcont.hu/progcont/100278/?pid=201288>

Homework 2

New curriculum model

- ▶ Write a program that reads an integer **N** from standard input. The next **N** lines each contain a subject code, a subject name, and a credit value in the following format:
 - ▶ `Subject_code:subject_name:credit_number`
 - ▶ The `subject_code` and `subject_name` are strings
 - ▶ `credit_number` is a positive integer.
 - ▶ The `subject_code` is always in the format "INBMM-xx-yy," where `xx` represents the recommended semester number, and `yy` is the unique identifier for the subject.
- ▶ Your program should print **only the subject names**, sorted in **ascending order** based on the recommended semester number. If multiple subjects belong to the same semester, they must be ordered by **increasing yy** (their unique identifier!)

Homework 2

New curriculum model

► Sample Input:

5

INBMM-02-07:Data Structures and Algorithms:6

INBMM-01-01:Algorithms and Fundamentals of Programming:2

INBMM-01-02:Electronics:6

INBMM-03-14:Fundamentals of Economics:6

INBMM-02-08:Mathematics for Engineers 2:6

Output for Sample Input:

Algorithms and Fundamentals of Programming

Electronics

Data Structures and Algorithms

Mathematics for Engineers 2

Fundamentals of Economics

Homework 3 – Club loyalty

- ▶ <https://progcont.hu/progcont/100360/?pid=201553>

Club loyalty

- ▶ Some athletes remain loyal to a single club throughout their entire sports career. Others are not like that: they change clubs from time to time to advance their careers or to improve their financial situation.
- ▶ Write a program that reads an integer **K** from the first line of standard input. After that, the input contains data about sports clubs, one per line, **until end-of-file (EOF)**.
- ▶ Each line contains the name of a club and the names of all athletes who have ever been associated with that club, in the following format:
`club_name:athlete_name[,athlete_name]...`
- ▶ Your program should print the names of all athletes who have been associated with **more than K different clubs**.
- ▶ If there are multiple such athletes, order them
 - first by the **number of clubs** they have been associated with (in **descending** order – more clubs comes first),
 - and if there is still a tie, by their **names in lexicographical ascending order**.
- ▶ The output format should match the sample output exactly.

Club loyalty

Sample Input

2

Golden Team:Grosics,Buzanszky,Lorant,Lantos,Bozsik,Zakarias,Kocsis,Hidegkuti,Puskas

Mighty Magyars:Buzanszky,Lantos,Zakarias,Hidegkuti

Marvellous Magyars:Grosics,Buzanszky,Bozsik,Hidegkuti,Puskas

Magnificent Magyars:Lorant,Budai,Kocsis,Hidegkuti,Puskas

Magical Magyars:Buzanszky,Lantos,Hidegkuti,Puskas

Output for Sample Input

Hidegkuti: 5

Buzanszky: 4

Puskas: 4

Lantos: 3